CSC 316

Lab 3: Encoding Data as Visual Channels

Where we are

This lab builds on your work with SvelteKit and `CLAUDE.md` in Lab 2. You will use Git to save working versions and trace how a component turns data into marks. You can complete this lab without a working interactive chart from Lab 2. We continue the **Frame → Generate → Judge** workflow and ask:

> **Which column of my data becomes which thing you can see?**

Mapping a data column to a visual channel, such as horizontal position, vertical position, size, or color, is called **encoding**. These choices affect which patterns a reader can see and how accurately they can compare values. An unsuitable encoding can obscure or misrepresent the data.

In this lab, you'll build **one chart**, adding **one encoding at a time**, and examine the code that draws it: **SVG primitives** (the shapes), **D3 scales** (the functions that turn data into pixels), and the line of code where a number in a CSV becomes a position on screen. The **data-to-pixels** exercise traces that conversion.

Learning Objectives

After completing this lab you will be able to:

- Name the **visual channels** (position, length, size, color, shape) and say which **data types** (quantitative, ordinal, categorical) each can represent
- Draw with **SVG primitives** (`circle` , `rect` , `line` , `text` , `g`) and reason about the **SVG coordinate system** (origin top-left, **y grows downward**)
- Explain a **D3 scale** as "a function from an input **domain** (data units) to an output **range** (such as pixels or colors)" and choose a scale **type** for each encoding: `scaleLinear` , `scaleSqrt` , `scaleSequential` , `scaleOrdinal`
- **Specify an encoding**, for example, "bubble map: `scaleLinear` x/y, `scaleSqrt` radius, `scaleSequential` color," so the prompt specifies how each variable should be represented
- Explain how a component receives data through **props**, renders one mark per row with `{#each}` , and uses a **D3 scale** to map a value to pixels
- Identify a **size encoding that misrepresents value ratios**, use `scaleSqrt` with a zero anchor when circle area should be proportional to value, and explain what a size legend does and does not communicate
- **Explain what your code does** in your process book (new **Question 6** this week)

Your hand-in: the Process Book

Continue using your **process book**, the Google Doc you **submit as a PDF each class**. Add a new section to the same doc, then export it as a PDF (**File → Download → PDF Document**) and upload it to the **Lab 3 assignment on Quercus** by **11:59 pm ET on Wednesday, October 7, 2026**.

Answer all six questions. Questions **1–5** document your process; **Question 6** asks you to trace what the code does.

```
LAB 3: Encoding Data as Visual Channels | <today's date>

1. WHAT WAS I GOING FOR?
   Include your encoding plan, written BEFORE generating the scatterplot,
   and your prediction sketch, drawn BEFORE generating the map.

2. WHAT DID I ASK FOR?
   Include your first scatterplot prompt word for word, not a summary.

3. WHAT CAME BACK?
   Include a screenshot of the first result, before you fixed anything.

4. WHAT DID I THINK OF IT?
   Explain whether it did what you asked. Answer the lab's check
   questions, including anything that did not work.

5. WHAT DID I CHANGE, AND WHY?
   Include your second prompt, second output, final screenshot, and a short
   explanation naming ONE idea from this week's lecture that informed
   a decision. Write the explanation in class without AI.

6. WHAT DOES THE CODE DO?
   Label the blocks: data in, transformation, output. Point at
   the {#each} that binds a row to a mark and a scale() call that
   turns a value into a pixel. Mark "?" on anything you can't name.
```

> **About Question 6.** You do not have to understand every line. Start by identifying the data, a component input, and something that appears on screen. In Activity 3, trace how a data value becomes a mark. Put `?` beside anything you cannot yet explain and discuss one question with the instructor or a tablemate.

Write it once, at the end. You do not need to write a reflection after every activity. **Capture** the materials you will need as you go (your encoding plan, your prompt, a screenshot, your prediction sketch), then do the whole write-up in **one pass at Activity 6**.

Prerequisites

You already have the toolkit from the previous lab (**Node.js**, **Claude Code**, a browser). If any of those aren't working, ask the instructor for help before you start. Use the setup handouts to review any installation steps.

Tool	Check it works
Node.js	Use the supported LTS release from Lab 2; ask the instructor if the project tools report an unsupported version.
Claude (Code)	`claude --version` prints a version
git	`git --version` prints a version
A browser	`http://localhost:5173` will open your app

> **New project this lab.** Keep your Lab 2 files and create a separate SvelteKit project. Working with one chart lets you examine each encoding in isolation. The earthquake dataset also has a different structure from the data in Lab 2. You will use the same tools and workflow with this new project.

A chart is a set of encoding choices

A chart maps data to visual channels: this column → this visual channel. A scatterplot says “column A → horizontal position, column B → vertical position.” A bubble chart adds “column C → size.” A choropleth says “region → shape on a map, value → color.”

The table below summarizes common channels and their uses. Position and length generally support more precise quantitative comparisons than area or color:

Channel	SVG attribute (roughly)	Data type	Considerations
Position (x, y)	`cx` / `cy`, `x` / `y`	quantitative; supports precise comparisons	use x and y for variables that require precise quantitative comparison
Length	`height` / `width` of a `rect`	quantitative, with a zero baseline	use a zero baseline when length represents the value
Size / area	`r` of a `circle`	quantitative magnitude	use **area to represent value**, rather than radius; see Activity 4
Color hue	`fill`	**categorical** (which group?)	use a small set of distinguishable hues and a colorblind-safe palette
Color value (light ↔ dark)	`fill`	quantitative (sequential)	a long tail can make most values appear similar
Shape	`<circle>` vs `<rect>` vs `<path>`	categorical, few values	use distinct shapes for a small number of groups

Two ideas guide this lab:

1. **A mark is an SVG primitive.** One earthquake is one `<circle>`. Its attributes (`cx`, `cy`, `r`, `fill`) control the channels. To encode a column into a channel, you set that attribute from that column.
2. **A scale maps values to a channel.** Data values and screen positions use different units. `mag` runs 6.0–8.8; a 600px canvas with 40px reserved for margins has a 560px plotting area. A **scale** is the function that converts one to the other:

```
const yScale = d3.scaleLinear()
  .domain([6, 8.8])      // input: data units (magnitude)
  .range([560, 0]);      // output: pixels (note: big value → small y → top)

yScale(8.8);  // 0    (top of the chart)
yScale(6.0);  // 560  (bottom)
```

The expression `cy={yScale(d.mag)}` maps a magnitude value to a vertical position. You choose which scale (`scaleLinear`? `scaleSqrt`? `scaleOrdinal`?) and which channel.

> **Specify the encoding in your prompt.** A request such as “Make it look nice” does not specify columns or scales. **Name the encoding** so the prompt states your decisions: “A scatterplot of depth (x, linear) vs magnitude (y, linear), one circle per quake.” This sentence specifies the chart type, columns, channels, and scales. You will evaluate whether the generated chart follows those instructions.

The Dataset: A year of magnitude 6.0+ earthquakes

The supplied file, `earthquakes_M6.0_1year.csv`, is a one-year snapshot of earthquakes of **magnitude 6.0 or greater** recorded worldwide. It contains **155 rows, one per earthquake**, from the **U.S. Geological Survey** (public domain). Check the README and `time` column for the reporting dates; the file does not update automatically. Put it in your project's `static/` folder so the dev server can serve it at `/earthquakes_M6.0_1year.csv`.

Data dictionary (the columns we'll encode)

Column	Meaning
`longitude`	Degrees East (−180 … 180). **A number** that maps to an x-scale.
`latitude`	Degrees North (−90 … 90). Maps to a y-scale; remember that SVG's y coordinates increase downward.
`depth`	Kilometres below the surface (5.6 … 636). Long right tail.
`mag`	Moment magnitude (6.0 … 8.8). **Logarithmic**; see the data constraints below.
`magType`	How magnitude was measured; mostly `mww`. Categorical.
`place`	Location description; can be shown in a tooltip.
`time`	UTC timestamp. Not a channel today (stretch goal).

Every value loads as a **string**; convert `longitude`, `latitude`, `depth`, `mag` to numbers on load.

Data constraints to check when evaluating your charts

- **Magnitude is logarithmic.** Each whole step (6→7→8) corresponds to approximately **32× more energy**; an 8.8 releases roughly **16,000×** the energy of a 6.0. The USGS explanation of magnitude and energy describes this relationship. Activity 4 separates three encoding decisions: **(1)** to make circle area proportional to a nonnegative value, radius must be proportional to its square root; `d3.scaleSqrt` supports this mapping; **(2)** using zero at the start of both its domain and range gives that proportional relationship; **(3)** area proportional to the magnitude number does not represent energy ratios. Include a **size legend (M6/M7/M8)** and state that size represents magnitude, not energy.
- **The file includes only M6.0+ earthquakes.** The catalog is filtered to M6.0+, so thousands of smaller quakes are **excluded by the filter**. State the M6.0+ filter so readers do not interpret the chart as a record of all earthquakes.
- **Depth has a long tail.** Most quakes are shallow (<70 km); a few are 300–636 km deep. A sequential color scale spanning the full range may make shallow earthquakes appear similar. Consider limiting the color domain and clamping values above its upper bound, or using a transformation. Explain any change in the legend.
- **Longitude wraps at ±180°.** Mapping longitude and latitude linearly at the same pixels-per-degree scale gives a plate carrée map. The Pacific is split at the left and right edges when longitude runs from −180° to 180°.

Patterns to examine

Plot `latitude` against `longitude` and look for **clusters near tectonic plate boundaries**, such as the Pacific Ring of Fire, the Mid-Atlantic Ridge, and the Andean and Aleutian subduction zones. This one-year M6.0+ sample may not show every boundary clearly. The two scales used for an abstract scatterplot can also place longitude and latitude on a map.

> **What pattern do you see?** Once your bubble map runs, look at where the quakes cluster and where the deep ones sit, and record one pattern you observe. **Your observation should be supported by the map**. Explain which locations or marks support it. (You'll note it in your process book.)

Activity 1: Scaffold the project

Here is the script to create a new project for this lab. (Your instructor may walk through it too; either way, run it yourself.)

Step 1: Create a SvelteKit project

In a terminal, inside your course folder (`cd ~/Documents/csc316-labs`):

```
npx sv create quake-encoding
#   answer the prompts (use arrow keys, then Enter):
#   • Which template?          SvelteKit minimal
#   • Add type checking (TS)?  No
#   • What to add to project?  nothing; press Enter to skip
#                              (Space toggles an add-on; Enter confirms/moves on)
#   • Which package manager?   npm         (this installs dependencies for you)
cd quake-encoding
npm install d3
npm run dev
```

Open the `http://localhost:5173/` link it prints. The page you'll edit is **`src/routes/+page.svelte`**. Leave this terminal running so the page updates when you save changes.

Step 2: Add the data

Copy `earthquakes_M6.0_1year.csv` into the project's **`static/`** folder. It's then served at **`/earthquakes_M6.0_1year.csv`**. Use that path with `d3.csv` .

> **Get the data: two files.** Both are in the course **download folder** and on **Quercus**. You can also **scan the code, or click / type the short link** beside it. Put `earthquakes_M6.0_1year.csv` in your project's `static/` folder; the README describes the data's structure and limitations (keep it in the project; Claude will use it too).
>
> **Download the data** `earthquakes_M6.0_1year.csv`
> Scan, or visit https://tinyurl.com/yfpapmcf
>
> **Download the data description** `earthquakes_README.md`
> Scan, or visit https://tinyurl.com/3s5mkpmb

Step 3: Save a starting version and open Claude

Open a second terminal and run `cd ~/Documents/csc316-labs/quake-encoding` . Leave the dev server running in the first terminal.

Save your work with Git. A commit saves a named version of your project that you can return to. If Git is not installed, follow **Install Git** at the start of the **Git & GitHub handout**, then check it with `git --version` . On a Mac, if Git asks to install the command line developer tools, click **Install** and ask the instructor if it takes more than a few minutes. The **Git & GitHub handout §§1–2** explains these first steps.

In a terminal in the project folder, outside Claude, run the following. Replace the name and email with your own before running the first two lines. If Git is already configured, keep your existing identity.

```
git config --global user.name "Your Name"
git config --global user.email "you@mail.utoronto.ca"
git init
git add -A
git commit -m "scaffold + data"
```

Start Claude in this second terminal:

```
claude
```

> **Save working versions.** Each time a chart works, inspect `git diff` in the terminal, then save with `git add -A` and `git commit -m "<describe the change>"`. You can ask Claude to show and explain the diff. Ask the instructor before discarding changes to restore an earlier version.

Activity 2: Write the project `CLAUDE.md` (standing context)

In Lab 2, you worked with `CLAUDE.md`. Create one for this project using the earthquake data and encoding requirements. Before asking Claude to build the chart, record the project and data rules in two steps:

1. **Run `/init`.** This Claude Code command inspects your project and scaffolds a starter `CLAUDE.md` describing the stack, file layout, and commands. It does not yet include your design decisions or the data constraints.
2. **Add the project rules.** Copy the block in **Appendix A** and paste it into Claude with: "Add these rules to our CLAUDE.md, merging them with what's already there." Claude edits the file; review the diff and accept it. (This gives you practice using the assistant to maintain its context file while reviewing the changes yourself.)

Include the **size-channel rules** from the dataset section: use `scaleSqrt`, anchor both domain and range at zero when area should be proportional to value, and show a magnitude legend. Recording these rules gives you a reference for checking the generated chart; it does not guarantee that Claude will follow them.

Confirm Claude read it: ask "According to our `CLAUDE.md`, what scale should encode magnitude as size, and why?" It should explain how `scaleSqrt` can map magnitude to circle area and why the domain and range matter.

Activity 3: Position first: an abstract scatterplot

We build the chart **one channel at a time**, starting with **position**, which supports precise comparisons. Begin with two quantitative columns mapped to x and y; you will use geographic coordinates later.

First, read one component input and loop

A page gives a component data through an input called a **prop**. For example, `<PointExample data={rows} />` passes the array `rows` as an input named `data`. Inside the component, `let { data } = $props();` receives that input. In an import path, `$lib` refers to `src/lib`.

This example uses `{#each}` to repeat markup for each input row. Read it before generating your chart; you do not need to add this demonstration to your project.

```
<!-- PointExample.svelte -->
<script>
  let { data } = $props();
</script>
<svg viewBox="0 0 200 100" aria-label="Example points">
  {#each data as d}
    <circle cx={d.x} cy="50" r="5" />
  {/each}
</svg>
```

A parent page can use `<PointExample data={[{x: 30}, {x: 80}, {x: 150}]} />` after importing the component. **Predict:** how many circles appear, and what changes if the middle row is removed? Explain to a tablemate how one input row becomes one mark. Here the x values are already pixels. In your chart below, D3 scales convert data units to pixels first.

After generating your chart, ask Claude: “Show me the lines where the component receives its data, and explain how the page supplies it.” Point to one mark on screen and the code that draws it.

The generated chart will also need to load and parse its CSV. Use the supplied loading requirements and trace the resulting numeric array into the component. You may see `$state` used while data loads; interactive state and derived selections are the focus of Lab 4. Keep this chart static.

Build the scatterplot

1. **Write the encoding prompt yourself.** In this activity, you will compose a prompt without a fill-in template. Specify four things: the **chart type**, the **columns**, the **channel** each column maps to, and the **scale type** for each. Write a prompt (jot it in your notes first, then paste it to Claude) that asks for a single scatterplot component in `src/lib/`, following your `CLAUDE.md`, and includes all of the following:
   - **x = depth, y = mag**, one `<circle>` per quake (fixed `r`, fixed fill; no variable size or color yet). For **each axis, name the scale type and why**. Remember SVG’s y grows downward, so one range is flipped.
   - Load `/earthquakes_M6.0_1year.csv` with `d3.csv` in `onMount`; convert `depth` and `mag` to numbers.
   - Use the D3 margin convention and axes generated with `d3.axisBottom` / `axisLeft`, **labelled with units** (you choose the text). Create axis `<g>` elements in Svelte and call the D3 axis generators on those elements.
   - Render the circles with `{#each}` in Svelte markup, using the scales for `cx` / `cy`. Use D3 for scales and axes, not for creating the earthquake circles. `d3.select` is allowed when attaching an axis to its `<g>` element.
   - Ask it not to add size or color yet; you will evaluate position first.

   Choose the scale types and labels yourself, and be prepared to explain those choices. If you need help with the wording, write a first attempt before asking the instructor.

> **Process book: capture this now.** Your encoding plan, including the columns, scale types, and labels chosen before generating the chart, is **Q1**; the exact prompt you wrote and ran is **Q2**. You’ll do the full write-up in one pass at Activity 6.

2. **Judge the position encoding.** Open the running page and check, criterion by criterion:
   - **Do higher magnitudes appear at the top?** Higher magnitudes should be at the **top**. If higher-magnitude earthquakes are at the bottom, the y-range is `[0, height]` instead of `[height, 0]`. Reverse the range to match the intended vertical order. (Recall: in SVG, y=0 is the top.)
   - **Are both axes labeled with units?** Use labels such as “depth (km)” and “magnitude,” rather than “x” and “y”.
   - **Is anything clipped at the edges?** If circles are half-cut on the border, leave enough room for the full circle radius by padding the range or plot boundaries. `.nice()` rounds domain bounds but does not guarantee enough space for the marks.

> **Process book: capture this now.** Save a screenshot of the first scatterplot before making any corrections. This is **Q3**.

3. **Read the code: trace data to pixels.** Ask Claude to show the component. Find these two things and say them out loud:
   - **The bind:** `{#each earthquakes as d}` repeats the markup so one row becomes one mark.
   - **The scale:** `cy={yScale(d.mag)}` is where a number becomes a pixel. Trace it: `d.mag` is `7.1`; `yScale` has domain `[6, 8.8]` and range `[height, 0]`; so `yScale(7.1)` is approximately `0.607 × height` from the top. This expression determines the mark's vertical position.

> **Why `{#each}` and not `d3.select().append()`?** D3 can create marks with `.data().enter().append()`, but in this course Svelte creates the earthquake marks. `{#each}` repeats the SVG markup for each row, and scales supply the attribute values. D3 axis generators are an exception: they draw ticks and labels inside dedicated `<g>` elements. See the D3 axis documentation for this pattern.

> **Reference handouts.** These companion handouts explain the concepts used here: **SVG: How a Chart Is Drawn** (the shapes and coordinate system), **D3, the Way We Use It** (scales and axes), and **Svelte Components** (`{#each}` and props). Refer to them when you encounter code you cannot yet explain.

> **Process book: capture this now.** This "read the code" step is **Q6**. Label the three blocks: data in (the `d3.csv` load + number conversion), transformation (the scales), output (the `{#each}` of `<circle>`s). Point at the `{#each}` and one `scale(d.field)` call. Mark `?` on anything you can't yet name.

Activity 4: Circle size and the data-to-pixels exercise

Now add a **third** channel: size. The following exercise separates three questions: how radius affects area, whether area is proportional to the encoded value, and what a magnitude value represents.

1. **First, create a version for comparison.** Run this prompt as written, then examine how the circle areas compare with the magnitude values:

```
Add a size channel: circle radius encodes mag. Use a LINEAR scale from mag to
radius (r from 3 to 30). Keep everything else the same.
```

Compare the M8.8 and M6.0 circles. **Circle area depends on radius squared, so radius ratios and area ratios differ.** With radii of 30 and 3, the area ratio is 100:1, while the magnitude-number ratio is about 1.47:1. Both the scale type and its endpoints affect this comparison. For a zero-anchored linear-radius mapping, doubling the value doubles the radius and quadruples the area.

2. **Claim 1: proportional circle area requires radius proportional to √value.** Because area equals πr^2, applying a **square root** to the value can give an area-proportional encoding. In D3, use `d3.scaleSqrt` and check the endpoints as described in Claim 2. **Write the revision prompt yourself:** ask Claude to use `d3.scaleSqrt` for radius and add a size legend showing M6, M7, and M8. Choose the **domain and range**: **anchor both at zero** for proportional areas, or use `[6,8.8] → [3,30]` to make differences easier to see while preserving only their order. State which you chose and why. Switching the scale type while keeping the same endpoint radii does not change the 100:1 area ratio between the smallest and largest marks.
3. **Claim 2: use a zero anchor for proportional areas.** `scaleSqrt` alone does not guarantee proportionality. Set the domain **and** range to start at zero: `domain([0, 8.8]).range([0, 30])`. The alternative `domain([6, 8.8]).range([3, 30])` is **not proportional**: it gives an **area ratio of 100×** across a **magnitude-number ratio of about 1.5×**. With a zero anchor, M8.8 has

about 1.5× the area of M6.0. The circles look similar because the numbers being encoded are close in ratio; this can make size a limited channel for comparing these magnitude values.

4. **Claim 3: magnitude ratios are not energy ratios.** A zero-anchored mapping makes area proportional to the magnitude number. M8 therefore has about 1.3× the area of M6, although the approximate energy ratio is 1,000:1. An area encoding of magnitude is not an encoding of energy. The **M6 / M7 / M8 size legend identifies the magnitude values** represented by the circles. Add a note that magnitude is logarithmic; a legend does not make the circle areas proportional to energy. Encoding an energy estimate instead is possible, but the large range would make smaller events difficult to see.
5. **Judge the difference.** Ask Claude to render the linear-radius version and your `scaleSqrt` version **side by side**. Compare both the ordering and the area ratios. Explain how the scale type and endpoints affect the result, including whether your chosen mapping preserves magnitude-number ratios.

> **Three points to check in a size encoding: (1)** proportional circle area requires radius proportional to the square root of the encoded value; **(2)** starting both domain and range at zero gives that relationship with `scaleSqrt`; **(3)** magnitude is logarithmic, so a size legend identifies magnitude values but does not turn their ratios into energy ratios. Use these distinctions to explain what your size channel represents and which comparisons it supports.

Activity 5: From scatterplot to map

Change the **two position columns** to turn the scatterplot into a map.

1. **Predict first, on paper.** Before generating the map, draw a blank rectangle with a roughly 2:1 aspect ratio and mark **where you expect M6.0+ quakes to cluster.** The rim of the Pacific? Down the middle of the oceans? Scattered evenly? Record your prediction on paper. Include the sketch in **Question 1** as work completed before asking AI to generate the map, then compare it with the result.
2. **Write the prompt to change the encoding.** Reuse the chart component and size scale. Specify the new position columns and add color and a basemap:
   - **which column becomes x** (the one that turns this into a map) and **which becomes y** (remember the reversed y range), both with linear scales;
   - keep `mag` → **size via** `scaleSqrt`;
   - add **color from a quantitative column of your choice**, via `d3.scaleSequential` (e.g. `d3.interpolateViridis`), with a color legend and a size legend;
   - set the map plotting area to **2:1**, with longitude domain `[-180, 180]` and latitude domain `[-90, 90]`, so both use the same pixels per degree;
   - draw a **faint world map (coastlines/land) behind** the points, aligned to the same longitude/latitude scales so the continents sit under the quakes;
   - axes are optional because the basemap provides geographic context.

   Choose the position and color columns yourself and state them in your prompt. Convert `longitude` and `latitude` to numbers before passing them to the component.

> **The basemap provides geographic context.** Ask Claude to use a small world-atlas dataset and draw the coastlines with the same longitude/latitude mapping as the points. The layers must share the projection, scale, and offsets. If they do not align, inspect those settings. Coastline shapes that cross ±180° must be split at the map boundary rather than drawn across the entire map.

> **Color here is a preview.** `scaleSequential` and the **Viridis** palette are the subject of the **Color** lecture later in the course. This activity introduces them before that discussion. For this activity, use a sequential scale with a perceptually even palette. The lecture will explain how these palettes support comparison.

3. **Compare the map with your prediction.** Look for events along the **Ring of Fire** around the Pacific and the **Mid-Atlantic Ridge** in the Atlantic. If you encoded depth as color, check whether deeper events occur **inland of ocean trenches**, where subducting plates descend. Use the legend to identify deep events. The 155-event sample may not show every pattern clearly, so a sparse pattern alone does not establish a coding error. Check that north is at the top and longitude and latitude have not been swapped.
4. **How the mapping works.** Longitude and latitude are quantitative columns that can be mapped to pixels with linear scales. Using the same pixels per degree for both produces a plate carrée projection. The basemap aligns when it uses the same mapping. **Position still uses the same encoding principle.** Changing the columns changes the meaning of the positions, while the component still draws one mark per row. Plate carrée distorts geographic area and distance, especially near the poles; later work will consider other projections.
5. **Check the color distribution.** If you chose depth, does the long tail make most points appear similar? Try a prompt such as "set the depth color domain's upper bound to the 95th percentile and enable clamping." Label the legend's upper end to show that it includes values at or above the cutoff. Compare the result with the full-range version and explain the loss of detail among deep events. If you chose another quantitative column, check its distribution and whether the scale distinguishes the values you want to compare.

Activity 6: Judge, then reflect

Run the **Frame → Generate → Judge** loop over the finished bubble map. Go channel by channel:

1. **Position matches the coordinates.** Latitude points north-up; longitude/latitude aren't swapped; marks and labels are not clipped.
2. **Size matches the stated mapping.** `mag` controls circle size via `scaleSqrt`. State whether **area is proportional to magnitude** or only ordered by it. Show M6/M7/M8 in the size legend and distinguish magnitude from energy.
3. **Color supports comparison.** Your chosen quantitative column uses a **sequential** scale with a labeled legend and appropriate units (km for depth). State any transformation or clamping and check that the values you want to compare remain distinguishable.
4. **The chart follows the frame.** The chart uses the channels you specified, and each channel represents the specified variable.
5. **Check the labels and caption.** The chart has a title, legends, and units where applicable. A caption acknowledges the filter: "M6.0 and larger only."

Turn each **mistake** into a revision request: "the depth legend has no units and the color runs light-on-white so shallow quakes vanish; label it 'depth (km)' and use a darker low end." **Ask Claude to judge its own chart against this list before you do.**

Now fill in your process book

You captured your encoding plan (Q1), your first prompt (Q2), your first-scatter screenshot (Q3), and your map-prediction sketch (also Q1) as you went, and labeled the code (Q6). Do the reflective part in **one pass**, using a few sentences for each response. Write it **in class without AI**:

- **Q1 / Q2 / Q3:** include your encoding plan, prediction sketch, first prompt, and first-scatterplot screenshot.
- **Q4 + Q5: your assessment (one short paragraph):** explain how the chart met the channel checks, including any failures, what you changed and why, and **one idea from this week's lecture** that informed a decision. Include your second prompt and output, and your final screenshot.

- **Q6: the code:** include the three labeled blocks and identify the `{#each}` and a scale call.
- **Your data story:** describe one pattern you observed and the map evidence that supports it.

Deliverable

At the end of class, export your Google Doc with the Lab 3 **process-book entry** and submit it as a PDF. It documents the chart you built, which should include:

- **A running SvelteKit project** that starts with `npm run dev` and has **no console errors**.
- **A project `CLAUDE.md`** capturing the earthquake data constraints (especially the size-channel rules).
- **A bubble-map component** that loads the CSV with `d3.csv` and encodes **four channels**: `longitude` → x, `latitude` → y, `mag` → size (`scaleSqrt`), your chosen quantitative column → color (sequential; depth is one option), with **legends and units**.

Also include the write-up for Activity 6, including all six answers and your data story:

1. **Q1 / Q2 / Q3:** include your encoding plan, prediction sketch, first prompt, and first-scatterplot screenshot.
2. **Q4 + Q5:** write one paragraph explaining how the chart met the channel checks, what you changed and why, and which lecture idea informed a decision. Include your second prompt and output, and your final screenshot.
3. **Q6:** include the labeled code blocks and identify the `{#each}` and a scale call.
4. **Your data story:** describe one pattern you observed and the evidence that supports it.

- **Optional: a stretch goal** (see below). If you complete one, document what you tried and how it went.

Export the doc as a PDF and upload it to the Lab 3 assignment on Quercus by 11:59 pm ET on Wednesday, October 7, 2026. Also **keep the project folder** for reference in later labs.

Stretch goals: optional

These activities are **optional**. If you choose to complete one, note what you did in your process book with a screenshot and a sentence or two:

- **Encode `magType` as shape** (circle vs. triangle vs. square) and evaluate whether readers can distinguish the groups. Compare shape with the other channels you used.
- **Add a tooltip** on hover showing `place` , `mag` , and `depth` ; this previews the interaction work in Lab 4.
- **Add time.** Color or animate by `time` (parse it with `d3.timeParse` / `new Date`). Do the big quakes cluster in bursts (aftershock sequences)?
- **Try a magnitude histogram** and examine the lower cutoff at M6.0. Are counts higher near that threshold? The filter excludes smaller earthquakes, so empty bins below M6.0 do not mean that no smaller events occurred.

Appendix A: `CLAUDE.md` starter (copy/paste, then edit)

```
# Quake Encoding: project context

## Stack
- SvelteKit (runs on Vite). Dev server: `npm run dev`, http://localhost:5173.
- Svelte 5 runes ($state, $derived, $props; not legacy `$:`).
- D3 for scales, axes, and CSV loading. Svelte creates the earthquake marks
  with {#each}. Do not create them with d3.select/append. D3 axis generators
  may use selections to draw inside dedicated axis <g> elements.

## Conventions
- One component per .svelte file in src/lib/; import with the $lib alias.
- The dataset lives in static/earthquakes_M6.0_1year.csv; load it client-side
  with d3.csv (in onMount) from /earthquakes_M6.0_1year.csv.
- Convert longitude, latitude, depth, mag to numbers on load.
- Use the D3 margin convention; label axes with units.

## Encoding rules
- Use d3.scaleSqrt for circle radius. For area proportional to the encoded
  nonnegative value, radius must be proportional to its square root.
- Set domain AND range to start at 0 for proportional areas. If using
  [6, 8.8] -> [3, 30] instead, state that size preserves order, not value ratios.
- Magnitude is logarithmic (each +1 ≈ 32× energy). Size represents magnitude,
  not energy. Include an M6/M7/M8 size legend and explain this distinction.
- Position (x/y) via scaleLinear. In SVG y grows downward, so the y range is
  [height, 0]; high latitude / high magnitude goes to the TOP.
- Use a sequential scale for the chosen quantitative color column
  (e.g. d3.scaleSequential + interpolateViridis). Depth has a long right tail.
  If limiting its domain, enable clamping and label the cutoff in the legend.
- magType is categorical → an ordinal scale if ever encoded.

## Data constraints
- M6.0+ only: the small quakes are filtered out. Don't imply quakes are rare.
- No nulls in longitude/latitude/depth/mag. Values load as strings; convert them.

## Definition of done
- npm run dev runs with no console errors.
- Four channels: longitude→x, latitude→y, mag→size (scaleSqrt), and the
  chosen quantitative column→color. State the color column in the legend.
- Legends and units on size and color; latitude points north-up.
```

Appendix B: The Design Critique Checklist

From now on we judge every chart using **one cumulative checklist**, the **Design Critique Checklist** handout, which adds sections as the course progresses. This lab uses **§3 · Encoding & channels**; continue **§1 (truthful & readable)** and Lab 2's checks of design intent and shared data scope. **§2 (components & reactivity)** becomes required in Lab 4. Use the following §3 items in this lab:

- ❒ **Every channel names a column.** For each of x, y, size, color, shape: which data column drives it, or is it a constant?
- ❒ **The scale type fits the data type.** Quantitative → linear/sqrt/log/sequential; categorical → ordinal.
- ❒ **Size is read as area.** Use `scaleSqrt`; anchor domain **and** range at zero if you want area proportional to value; include a size legend and explain whether size represents magnitude or energy.
- ❒ **Check vertical position.** Reverse the y-range (`[height, 0]`) so higher values appear at the top; check for clipping at the edges.
- ❒ **Choose channels for the intended comparison.** Use position for precise quantitative comparisons and evaluate whether color or shape serves your other variables.
- ❒ **Readers can interpret the encodings.** Legends and axis units let a reader decode every channel.

- ❑ **The mappings support the stated comparisons.** Length uses a zero baseline; size states whether area is proportional to value; the color scale distinguishes the values being compared and discloses any clamping.
- ❑ **You can point to the binding and the scale in the code** (`{#each}` and a `scale(d.field)` call).

Appendix C: Troubleshooting

Symptom	Likely cause & fix
Higher-magnitude earthquakes appear at the **bottom** of the scatterplot	y-range is `[0, height]`. Flip it to `[height, 0]`; SVG y grows downward.
The map is upside down	Same y-flip, for `latitude`. North should map to the top → range `[height, 0]`.
Circles are half-clipped on the edges	Allow room for the largest radius using range padding or plot margins. `.nice()` rounds domain bounds but does not guarantee enough room.
One circle is much larger than the others	Check the radius scale and its endpoints. For proportional areas, use `d3.scaleSqrt` with both domain and range starting at zero. Changing the scale type alone does not change endpoint area ratios.
The whole map is one color except a few dots	Depth's long tail may compress the colors of most values. Try an upper domain bound at the 95th percentile with clamping enabled, and label the cutoff.
Chart is empty / "failed to fetch"	The CSV must be in `static/` and requested as `/earthquakes_M6.0_1year.csv` (leading slash, no `static/`).
Circles all sit at `cx=NaN`	Check for missing values, failed numeric conversions, an incorrect field name, or an invalid scale domain. Parse numeric columns on load, for example with `+d.longitude`.
Page didn't update	Check the dev-server terminal for a compile error; fix it and it hot-reloads.